第 15 章：Protocol、ABC 与实用泛型

先记住一句话

`Protocol` 说明“对象要具备什么能力”，ABC 说明“子类必须继承什么骨架”，泛型说明“几个类型位置之间必须保持什么关系”。

这三类工具经常一起出现在 Agent 框架中。模型客户端、搜索后端和工具实现需要被替换；框架内部可能需要统一生命周期；一个工具的输入类型又必须与成功输出类型一路传到注册表调用端。

本章围绕一个同步搜索工具讲清这些问题。学完后，你应该能：

- 判断一个需求该用普通函数、`Protocol` 还是 ABC；
- 解释结构化子类型与 Java `implements` 的区别；
- 说明 `@runtime_checkable` 为什么不是运行时类型校验；
- 从 `identity[T]`、`Result[T]` 读出类型参数连接了哪些位置；
- 逐项读懂 `Tool[SearchRequest, list[SearchHit]]`；
- 区分 union、constraint（约束）与 bound（上界）；
- 为只读参数选择 `Mapping` / `Sequence`，并解释 `list` 的不变性。

本章只讨论同步、本地且可控的例子，不涉及大型依赖注入容器、元类或并发工具调度。

15.1 Java → Python 心智模型

Java 中熟悉的概念	Python 中可以先想到	最重要的不等价之处
接口与 `implements`	`Protocol`	实现类通常不用声明继承； 检查器按成员结构判断兼容性
抽象类与抽象方法	`ABC`、`@abstractmethod`	没有编译阶段强制； 不完整子类在实例化时被拒绝
只靠约定调用某个方法	duck typing	Python 直接查找并调用成员， 不先确认接口身份
	`isinstance(x,`	运行时协议只做浅层成员检查，

7127 2502x6dg8fpjJ0sff400d0gp/1/crossnote=1=a48d6ce6e1cdd5aaa8662ffa8b.html

`instanceof SomeInterface`	`isinstance(x, RuntimeProtocol)`	运行时协议只做浅层成员检查， 不核对完整签名
`<T> T identity(T value)`	`def identity[T](value: T) -> T`	注解主要服务静态分析， 不插入运行时检查或转换
`Function<A, B>` 等函数式接口	`Callable[[A], B]`	兼容的函数或可调用对象自动满足， 不需要声明实现
带复杂方法签名的函数式接口	定义 `__call__` 的 `Protocol`	可以表达关键字参数名等规则， 仍是结构化匹配
`List<T>` / `Map<K, V>`， 只读视图	`Sequence[T]` / `Mapping[K, V]`	限制接口可见能力， 不保证底层对象真的不可变
泛型 sealed result	成功为 `Success[T]`，失败为 `Failure`	union 类型别名不是运行时密封层次， 注解也不会验值
Bean Validation / 反序列化校验	显式校验、Pydantic 等	`Protocol` 、ABC 和泛型都不能替代外部数据校验

可以把 Pyright 类比成项目额外执行的一道编译期检查，但不能完全等同于 `javac`：即使类型检查没有通过，Python 解释器仍然可以直接运行源码。类型提示不会自动改变运行时对象。

15.2 先看完整数据流

本章最终组装出的关系如下：

```
MemorySearchBackend ─┐
FixtureSearchBackend ─┼─► 满足 SearchBackend ─► 被 SearchTool 依赖 ─┐
OfflineSearchBackend ─┘                                              │
                                                                     ├─► ToolRegistry
empty_search_tool ──────────────► 直接满足 Tool 协议 ────────────────┘

SearchRequest ─► registry.invoke(...) ─► 选中的工具 ─► Result[list[SearchHit]]
```

这里有三层不同的约束：

1. `SearchBackend` 只描述所有搜索后端都要提供的能力；
2. `Tool[InputT, OutputT]` 把输入类型与成功输出类型带过调用边界；
3. `ToolRegistry[InputT, OutputT]` 保证同一个注册表中的工具使用同一组输入、输出类型。

这里有三层不同的约束：

1. `SearchBackend` 只描述所有搜索后端都要提供的能力；
2. `Tool[InputT, OutputT]` 把输入类型与成功输出类型带过调用边界；
3. `ToolRegistry[InputT, OutputT]` 保证同一个注册表中的工具使用同一组输入、输出类型。

运行时仍然只有普通对象、属性查找、函数调用和异常。图中的类型关系主要由 Pyright 检查。

15.3 Duck typing：先用能力，不先问类名

Python 调用方法时，默认不会先查对象是否声明实现某个接口：

```
class MemoryBackend:
    def search(self, query: str, *, limit: int) -> list[str]:
        return [query][:limit]


backend = MemoryBackend()
hits = backend.search("Protocol", limit=1)
```

解释器只会在 `backend` 上查找 `search`，按参数规则完成绑定，然后执行方法。对象有合适的成员，调用就可能成功；缺少成员或参数不匹配，错误通常到实际执行这条路径时才出现。

这就是 duck typing。它适合轻量实现和测试替身，但“能调”不等于“契约已经表达清楚”。公共 API 如果只依赖口头约定，IDE 和静态检查器就难以及早发现拼错方法名、传错参数或返回错误类型。

`Protocol` 的作用，是把这份已有的能力约定写成可静态检查的结构。

15.4 Protocol：Python 的结构化接口

先定义搜索结果和调用方真正需要的接口：

```
from collections.abc import Sequence
from dataclasses import dataclass
from typing import Protocol


@dataclass(frozen=True)
class SearchHit:
```

```python
@dataclass(frozen=True)
class SearchHit:
    title: str


class SearchBackend(Protocol):
    def search(self, query: str, *, limit: int) -> Sequence[SearchHit]: ...
```

这个协议声明了四件事：

- 对象要有名为 `search` 的成员；
- `query` 应接收 `str`；
- `limit` 必须以 `limit=...` 的关键字形式传入，并且类型为 `int`；
- 调用方只依赖一个可顺序读取的 `SearchHit` 序列。

实现类不必写 `class Xxx(SearchBackend)`：

```python
class MemorySearchBackend:
    def search(self, query: str, *, limit: int) -> list[SearchHit]:
        return [SearchHit(title=query)][:limit]


backend: SearchBackend = MemorySearchBackend()
```

Pyright 会比较成员结构和签名。`list[SearchHit]` 提供了 `Sequence[SearchHit]` 承诺的只读能力，所以这个更具体的返回值可以满足协议。通过协议使用结果时，调用方只能依赖协议承诺的 `Sequence`，不能擅自假定具体实现一定返回 `list`。

这叫结构化子类型。Java 接口通常采用名义类型：类要显式 `implements SearchBackend` 才属于该接口。Python `Protocol` 更像“只要形状兼容就算满足”，因而特别适合：

- 接入无法修改继承关系的第三方对象；
- 为测试写一个很小的 fake；
- 让多个互不相关的类暴露同一项能力；
- 避免调用方依赖具体后端名称。

协议也不会向隐式实现类复制代码。即使协议方法写了默认实现，没有继承协议的类也不会得到那段方法体。要复用行为，应使用组合、普通辅助函数或 ABC。

README × | readme × | README × +
z72502x6dg8fpjj0sff40000gp/T/crossnote-f-a48d6ce6e1cdd5aaa8662ffa8b.html
eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家

15.5 `runtime_checkable` 只检查“有没有成员”

普通自定义协议不能直接用于 `isinstance()`。加上 `@runtime_checkable` 后可以检查所需成员是否存在：

```
from typing import runtime_checkable


@runtime_checkable
class SearchBackend(Protocol):
    def search(self, query: str, *, limit: int) -> Sequence[SearchHit]: ...
```

但这不是 Java `instanceof` 接口类型，也不是反射式的完整签名验证。考虑这个故意写错的实现：

```
class WrongBackend:
    def search(self, query: int, *, limit: str) -> list[SearchHit]:
        return []
```

下面三个结论可以同时成立：

检查	结果	原因
`isinstance(WrongBackend(), SearchBackend)`	`True`	运行时找到了 `search` 成员
某次动态调用	可能碰巧成功	注解不会在调用时验证实参
赋给 `SearchBackend` 变量并运行 Pyright	报错	静态检查会比较参数与返回签名

因此，运行时协议通过不能证明参数个数、参数名、类型、返回值或业务行为正确。它适合在动态加载插件后做一次浅层能力探测，不能代替 Pyright，也不能代替对 JSON、HTTP 或 LLM 输出的运行时校验。

15.6 ABC：需要共享骨架时再要求继承

ABC 是 abstract base class（抽象基类）。它适合表达“子类必须参与这个继承体系，并复用统一流程”：

```
from abc import ABC, abstractmethod
```

ME × | README × | readme × | README × +
olders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-a48d6ce6e1cdd5aaa8662ffa8b.html
期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

15.6 ABC：需要共享骨架时再要求继承

ABC 是 abstract base class（抽象基类）。它适合表达"子类必须参与这个继承体系，并复用统一流程"：

```
from abc import ABC, abstractmethod
from collections.abc import Mapping


class ManagedTool(ABC):
    def __init__(self) -> None:
        self._call_count = 0

    def run(self, payload: Mapping[str, object]) -> str:
        self._call_count += 1
        return self.execute(payload)

    @abstractmethod
    def execute(self, payload: Mapping[str, object]) -> str: ...
```

具体子类继承 `ManagedTool` 后，会真正获得 `__init__()` 、计数状态和 `run()` 模板流程。若子类没有实现 `execute()` ，类定义可以完成，但实例化时会抛出 `TypeError` 。

这与 Java 抽象类解决的是同一类问题，但约束时机不同：Java 编译器通常在编译阶段拒绝不完整具体类；Python 的 `ABCMeta` 主要在运行时创建实例时拦截。 `@abstractmethod` 也不负责检查传入字典的字段。比如 `payload["text"]` 必须为字符串，仍要在 `execute()` 的 I/O 边界显式判断。

函数、Protocol 与 ABC 怎么选

实际需求	优先选择	原因
只有一个无状态动作	普通函数 + 精确 `Callable`	依赖最少，调用关系直接
多个无关对象只需提供同一能力	`Protocol`	按结构兼容，不强迫继承
第三方类或测试替身不能改基类	`Protocol`	实现方无需知道协议存在
子类必须复用状态或模板流程	ABC	继承会真正带来代码和状态
不完整子类必须在实例化时失败	ABC + `@abstractmethod`	提供运行时实例化约束

外部值要逐字段验证	显式校验或验证库	这不是接口抽象负责的问题

`Protocol` 不是 ABC 的新版本，ABC 也不是更严格的 `Protocol`。一个项目完全可以对外暴露协议，在框架内部用 ABC 管理受控生命周期。

15.7 泛型表达的是关系，不是“什么都能传”

先看一个丢失关系的签名：

```
def accept_unknown(value: object) -> object:
    return value
```

它允许传入 `SearchHit`，但静态返回类型只是 `object`。调用方不能直接访问 `.title`，因为签名没有说明返回值与输入值属于同一类型。

泛型把同一个类型变量放进多个位置：

```
def identity[T](value: T) -> T:
    return value


number = identity(15)  # T 求解为 int，结果是 int
text = identity("search")  # T 求解为 str，结果是 str
```

这与 Java 的 `<T> T identity(T value)` 很接近：每次调用都求出合适的 `T`，参数与返回位置保持关系。它不等于 `Any`：`Any` 会大幅放弃检查，而 `T` 恰恰要求相关位置保持一致。

运行时，`identity()` 只是原样返回同一个对象。`T` 不会复制、转换或验证这个对象，也不能写 `isinstance(value, T)`。泛型参数主要供静态工具使用。

容器元素也能参与关系

```
from collections.abc import Sequence


def first[T](values: Sequence[T]) -> T:
    if not values:
        raise ValueError("first() 不能接收空序列")
```

容器元素也能参与关系

```
from collections.abc import Sequence


def first[T](values: Sequence[T]) -> T:
    if not values:
        raise ValueError("first() 不能接收空序列")
    return values[0]
```

`Sequence[T]` 中的 `T` 是元素类型，`-> T` 把这个类型带到返回值：传入 `tuple[str, ...]` 得到 `str`，传入 `list[SearchHit]` 得到 `SearchHit`。

类型关系不能凭空制造一个元素。空序列没有 `T` 可以返回，因此 API 必须选择明确策略：抛异常、返回 `T | None`，或改用能表达“非空”的数据结构。本章示例选择抛出 `ValueError`。

15.8 用 `Result[T]` 保留成功值的具体类型

工具失败有时是正常业务分支，调用方希望把成功与失败都作为数据处理。可以定义：

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Success[T]:
    value: T


@dataclass(frozen=True)
class Failure:
    error: Exception


type Result[T] = Success[T] | Failure
```

`Result[list[SearchHit]]` 表示：成功时 `value` 精确为 `list[SearchHit]`，失败时保留原异常对象。调用方通过 `isinstance(result, Success)` 判断后，Pyright 会把联合收窄到成功分支。

它可以先类比为 Java 中 `Result<T>` 的成功/失败 sealed 层次，但不能完全等同：这里的

README × | README × | readme × | README × +
wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-a48d6ce6e1cdd5aaa8662ffa8b.html
旧关... | eclp相关 | 配运运单26年需求... | 运单小秘值班安排 | 研发日常任务 | 运单运维 | 运单-资源大屏 | 20260618-应急预案 | 运单26年需求迭代... | 商家下单

它可以先类比为 Java 中 `Result<T>` 的成功/失败 sealed 层次，但不能完全等同：这里的 `Result[T]` 只是类型别名，不会生成新的运行时包装类，也不会阻止未检查的代码传入别的对象。由于泛型信息不用于这种运行时分支，也不要写 `isinstance(result, Success[int])`；检查实际类 `Success` 即可。

`Result[T]` 更不意味着捕获一切。示例中的 `SearchTool` 只把预计会发生的 `OSError` 和 `ValueError` 变成 `Failure`，并保留原始异常。未预期的程序缺陷应继续暴露，退出或中断信号也不应为了得到漂亮返回类型而被吞掉。如果现有调用栈已有统一异常处理，直接抛异常可能更合适。

15.9 新旧泛型语法都要会读

本项目要求 Python 3.14，主写法采用 Python 3.12 引入的类型参数语法：

```
class Box[T]:
    def __init__(self, value: T) -> None:
        self._value = value

    def get(self) -> T:
        return self._value
```

兼容 Python 3.11 及更早版本的框架常用传统写法：

```
from typing import Generic, TypeVar

LegacyT = TypeVar("LegacyT")


class LegacyBox(Generic[LegacyT]):
    def __init__(self, value: LegacyT) -> None:
        self._value = value

    def get(self) -> LegacyT:
        return self._value
```

两者表达同一关系：构造时保存的类型与 `get()` 返回类型一致。

问题	传统写法	Python 3.12+ 写法
类型变量声明位置	模块级 `TypeVar("LegacyT")`	函数名或类名后的 `[T]`

两者表达同一关系：构造时保存的类型与 `get()` 返回类型一致。

问题	传统写法	Python 3.12+ 写法
类型变量声明位置	模块级 `TypeVar("LegacyT")`	函数名或类名后的 `[T]`
泛型类标记方式	继承 `Generic[LegacyT]`	`class Box[T]` 已隐式声明泛型
泛型函数写法	在注解中复用模块级 `T`	`def identity[T](...)`

不要混写 `class Box[T](Generic[T])`。新语法已经建立泛型参数，不需要再继承 `Generic[T]`。

15.10 Union、constraint 与 bound 解决不同问题

这三类写法都在限制类型，但问题不同。

Union：一个位置可能是哪几种类型

```
def passthrough(value: str | bytes) -> str | bytes:
    return value
```

`str | bytes` 只承诺值属于这个集合。返回注解仍是整个 union，没有表达“返回类型一定和本次输入的具体类型相同”。

Constraint：多个位置要选同一个有限候选

```
def combine[A: (str, bytes)](left: A, right: A) -> A:
    return left + right
```

一次调用会从 `str`、`bytes` 中为 `A` 选择同一个候选。两个字符串可以，两个字节串也可以；`combine("tool-", b"result")` 会被 Pyright 拒绝。传入 `str` 的子类时，结果通常归一为约束中列出的 `str`。

Bound：至少具备某项能力，同时保留具体子类型

```
class HasTitle(Protocol):
    @property
    def title(self) -> str: ...
```

Bound：至少具备某项能力，同时保留具体子类型

```
class HasTitle(Protocol):
    @property
    def title(self) -> str: ...


def first_titled[T: HasTitle](values: Sequence[T]) -> T:
    item = first(values)
    if not item.title:
        raise ValueError("title 不能为空")
    return item
```

`T: HasTitle` 表示 `T` 的上界是 `HasTitle`。函数体可以安全读取 `.title`，但返回值仍保留调用方传入的具体类型，例如 `SearchHit`，不会退化成 `HasTitle`。

写法	要回答的问题	多位置关系	结果类型特点
`str 或 bytes`	这个值可能是哪几种类型?	不自动关联	仍是 union
`A: (str, bytes)`	本次调用从有限候选选哪一个?	同一个 `A` 贯穿各位置	归一到候选类型
`T: HasTitle`	值至少要具备什么能力?	同一个 `T` 贯穿各位置	保留具体子类型

传统语法分别写作 `TypeVar("A", str, bytes)` 和 `TypeVar("T", bound=HasTitle)`。同一个类型变量不能同时拥有 constraints 和 bound。

15.11 `Callable` 与 `Protocol.__call__`

对于固定的位置参数和返回值，`Callable` 最直接：

```
from collections.abc import Callable, Sequence

type HitFormatter = Callable[[Sequence[SearchHit]], str]


def render(hits: Sequence[SearchHit], formatter: HitFormatter) -> str:
    return formatter(hits)
```

ME × README × readme × README × +
olders/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-a48d6ce6e1cdd5aaa8662ffa8b.html
期接的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代...

它可以先类比为 Java 的 `Function<List<SearchHit>, String>` 。但 Python 不要求函数或对象声明实现某个函数式接口；只要调用签名兼容，就能传给 `render()` 。

基础 `Callable` 不适合表达关键字参数名等复杂规则。这时定义带 `__call__` 的协议：

```
class PrefixedFormatter(Protocol):
    def __call__(self, hits: Sequence[SearchHit], *, prefix: str) -> str: ...
```

普通函数 `format_hits(hits, *, prefix=...)` 可以直接满足该协议。如果实现把关键字参数命名为 `label` ，调用方按协议传 `prefix=...` 就会失败，Pyright 因而会报告签名不兼容。

下一章的装饰器还需要保留“任意参数列表”的完整关系，那会用到 `ParamSpec` 。本章先记住：一个普通 `T` 代表一个值类型，不能代表任意长的参数列表。

15.12 把关系带进 `Tool` 和注册表

工具协议有两个类型参数：

```
class Tool[InputT, OutputT](Protocol):
    def __call__(self, tool_input: InputT, /) -> Result[OutputT]: ...
```

逐项代入 `Tool[SearchRequest, list[SearchHit]]` ：

签名位置	代入后的类型	含义
`InputT`	`SearchRequest`	调用时接收搜索请求
`OutputT`	`list[SearchHit]`	成功值是具体命中列表
`Result[OutputT]`	成功为 `Success[list[SearchHit]]` ，否则为 `Failure`	失败不抹掉成功值类型
`__call__`	对象可以像函数一样调用	类实例和普通函数都可能满足协议

`/` 表示 `tool_input` 是仅限位置参数，调用时写 `tool(request)` ，不能写 `tool(tool_input=request)` 。

注册表继续复用同一组类型变量：

```
class ToolRegistry[InputT, OutputT]:
    def register(self, name: str, tool: Tool[InputT, OutputT]) -> None: ...

    def invoke(self, name: str, value: InputT) -> Result[OutputT]: ...
```

本章采用同构注册表：同一个实例里的所有工具都接收 `SearchRequest`，并产生 `Result[list[SearchHit]]`。因此工具实现、注册、调用和结果处理之间的关系不会退化成 `Any`。

真实 Agent 注册表常常同时保存搜索、天气、数据库等输入输出完全不同的工具。一个普通 `dict[str, Tool[object, object]]` 无法让检查器仅凭运行时字符串恢复每个名字对应的类型。实际项目可能使用 `Literal` + 重载、代码生成、运行时模型校验，或在一个明确边界做受控类型擦除。不要为了把异构工具塞进字典，就宣称 `Tool[Any, Any]` 仍然保留了类型安全。

15.13 为什么只读参数优先 `Mapping` / `Sequence`

公共参数应只索取实现真正需要的能力：

```
def first[T](values: Sequence[T]) -> T: ...


class ToolRegistry[InputT, OutputT]:
    def __init__(self, tools: Mapping[str, Tool[InputT, OutputT]]) -> None:
        self._tools = dict(tools)
```

`first()` 只读取长度和下标，因此不需要调用方必须提供 `list`。注册表构造器只读取初始映射，所以接收 `Mapping`；内部确实要增删条目时，再复制为自己拥有的 `dict`。

这与 Java “参数面向所需的最小集合接口”思路相似，但 `Sequence` / `Mapping` 并不承诺深度不可变，也不取得对象所有权。别处仍可能修改底层列表或字典。若必须保存稳定快照，需要显式复制。

为什么 `list[SearchHit]` 不能当成 `list[object]`

```
def append_marker(values: list[object]) -> None:
    values.append("debug-marker")


hits: list[SearchHit] = []
```

为什么 `list[SearchHit]` 不能当成 `list[object]`

```
def append_marker(values: list[object]) -> None:
    values.append("debug-marker")


hits: list[SearchHit] = []
# append_marker(hits)  # Pyright 拒绝
```

如果允许传入， `append_marker()` 有权向列表加入任意 `object` ，于是原本保证只含 `SearchHit` 的 `hits` 被混入了 `str` 。因此可变 `list` 的元素类型默认是不变的。

只读的 `Sequence` 没有 `append()` ， `Sequence[SearchHit]` 可以安全交给只按 `Sequence[object]` 读取的消费者，这称为协变。完整协变/逆变推导属于进阶主题；当前先使用一条工程规则：只读 API 优先公开只读抽象，确实要修改时才接收准确元素类型的具体可变容器。

15.14 Java 工程师最容易踩的坑

1. **把 `Protocol` 当成必须实现的 Java 接口。** 这样会丢掉结构化匹配的主要价值。需要共享代码或强制继承时，需求才更接近 ABC。
2. **把 `@runtime_checkable` 当成完整反射校验。** 它只确认成员存在，不验证参数和返回值。
3. **把 `Any` 当成泛型。** `Any -> Any` 放弃关系； `T -> T` 保留关系。
4. **认为泛型注解会在运行时验值。** Python 不会因 `Box[int]` 自动阻止字符串进入对象。
5. **把 constraint 写成 union。** Union 表达候选集合，共享的 constraint 才表达多个位置要选择同一家族。
6. **让 `Result[T]` 捕获所有异常。** 只把调用方计划处理的失败变成数据，并保留原异常。
7. **用 `Tool[Any, Any]` 掩盖异构注册表。** 代码能装进字典，不代表输入输出关系仍被检查。
8. **把 `list[具体类型]` 放宽成 `list[object]` 。** 接收方可以写入其他对象，破坏原列表保证。
9. **把接口注解当外部数据验证。** JSON、HTTP、数据库和 LLM 输出仍需运行时校验。

15.15 运行本章项目

本章只使用标准库，没有新增运行依赖。仓库根 `pyproject.toml` 要求 Python `>=3.14` ；这使 `def identity[T](...)` 等 Python 3.12+ 语法可以直接运行。

本章文件分工：

本章文件分工：

文件	用途
`README.md`	独立教程与练习
`example.py`	六组可从头运行的同步实验
`search_fixture.json`	固定、只读的本地搜索数据
`static_errors.py`	故意保留的静态类型错误，只交给 Pyright

从仓库根目录运行正常示例：

```
.venv/bin/python -B java_to_python/chapters/15_protocols_generics/example.py
```

`-B` 只用于阻止生成 `.pyc`。现有 `java_to_python/run_all.py` 只支持前 10 章，因此本章单独运行。输出分为六组，重点观察：

- `identity()`、`first()` 和 `Result[T]` 如何保留具体类型；
- 内存后端与 fixture 后端如何通过同一个 `SearchBackend` 被调用；
- 错误签名为何能通过浅层 `isinstance()`，却不能通过静态检查；
- 类实例工具与普通函数如何进入同一个注册表；
- `Callable`、`__call__` 协议和 ABC 各自解决什么问题；
- 绕过列表不变性检查后，`SearchHit` 列表如何被混入 `str`。

fixture 路径由 `Path(__file__)` 定位，不依赖启动时工作目录。加载器会先验证 JSON 顶层数组、每行对象和四个必需字符串字段，再创建 `SearchHit`。示例不联网、不需要 API Key，也没有后台线程或文件写入。它不能模拟真实后端的超时、鉴权、分页、连接池和取消行为。

15.16 同时验证运行行为和静态关系

正常示例应该通过 Pyright：

```
.venv/bin/pyright --project pyproject.toml \
  java_to_python/chapters/15_protocols_generics/example.py
```

故意错误文件应该以非零状态退出：

```
.venv/bin/pyright --project pyproject.toml \
  java_to_python/chapters/15_protocols_generics/static_errors.py
```

当前 `static_errors.py` 预期产生 **7 个错误、4 条类型信息**，分别覆盖协议签名、`identity()`、`first()`、工具输出、关键字回调、列表不变性、constraint，以及四个 `reveal_type()` 结果。不要直接执行这个文件；`reveal_type()` 和故意错误是静态实验。

代码风格检查：

```
.venv/bin/ruff check java_to_python/chapters/15_protocols_generics
.venv/bin/ruff format --check java_to_python/chapters/15_protocols_generics
```

本次使用 CPython 3.14.7、Pyright 1.1.411 和 Ruff 0.16.5 验证：正常示例完整运行，主文件为 0 个类型错误，故意错误文件得到预期 7 个错误和 4 条类型信息，Ruff 两项检查均通过。

运行通过只证明当前真实路径可以执行；Pyright 通过只证明静态签名关系满足当前规则；Ruff 通过只证明代码符合配置的格式和常规检查。三者不能互相替代。

15.17 递进练习

建议另建练习文件，不要修改 `static_errors.py` 的预期诊断数量。

练习 1：增加一个结构化后端

- **要求**：实现 `PrefixSearchBackend`，只匹配标题前缀，不显式继承 `SearchBackend`。
- **输入**：两个 `SearchHit`，查询 `"Protocol"`，分别设置 `limit=1` 和 `limit=2`。
- **验收**：能赋给 `SearchBackend` 变量并通过 Pyright；调用方不按类名分支；空匹配返回空序列。

练习 2：组合泛型结果

- **要求**：实现 `map_success[T, U](result: Result[T], transform: Callable[[T], U]) -> Result[U]`。
- **输入**：`Success(3)` 加整数转字符串函数，以及 `Failure(ValueError("bad"))`。
- **验收**：成功结果推断为 `Result[str]`；失败前后的异常是同一个对象；不使用 `Any` 或 `cast`；转换函数抛出的异常不被悄悄吞掉。

练习 3：比较 Protocol 与 ABC

cast ；转换函数抛出的异常不被悄悄吞掉。

练习 3：比较 Protocol 与 ABC

- **要求**：定义只要求 close() 的 Closable 协议，再定义带共享 closed 状态的 ManagedResource(ABC) ，各写一个具体实现。
- **输入**：正常关闭两次，并尝试实例化一个缺少抽象方法的子类。
- **验收**：现有对象可以结构化满足协议；ABC 子类继承共享状态；不完整子类实例化得到 TypeError ；能解释两种设计分别解决什么。

练习 4：选择 union、constraint 或 bound

- **要求**：分别为“原样返回 str | bytes ”“合并两个同编码家族的值”“从有 name 属性的序列取第一项并保留具体类型”设计签名。
- **输入**： str 、 bytes 、 str 子类，以及两个具有 name 属性的不同数据类。
- **验收**：Pyright 拒绝混合文本/字节合并； reveal_type() 能观察 constraint 的归一结果和 bound 保留的具体类型；能解释为什么不能全部改成 union。

练习 5：扩展同构注册表

- **要求**：增加 names() -> Sequence[str] 和 unregister(name: str) -> Result[Tool[InputT, OutputT]] 。
- **输入**：注册两个搜索工具，删除一个，再删除未知名字。
- **验收**：名字顺序和删除结果可观察；取回的工具仍保留输入输出关系；未知名字保存 LookupError ；不使用 Tool[Any, Any] 。

15.18 本章小结

- Duck typing 是 Python 的运行方式； Protocol 把隐含能力写成静态结构契约。
- @runtime_checkable 只做浅层成员探测，不验证完整签名或业务行为。
- ABC 通过显式继承复用状态和流程，并在实例化时拒绝未完成抽象方法的子类。
- 泛型的核心是保留参数、容器元素、回调和返回值之间的类型关系，不是接受任意值。
- Union 表达可能集合，constraint 表达有限候选中的同一选择，bound 表达最低能力并保留子类型。
- 简单回调用 Callable ；要表达关键字名等复杂调用规则时，用带 __call__ 的 Protocol 。
- Mapping / Sequence 适合只读消费者；可变 list 不能随意放宽元素类型。
- 静态类型无法替代运行时校验。来自 JSON、HTTP、数据库和 LLM 的数据始终要在边界验证。

```python
"""第 15 章：用 Protocol、ABC 与泛型组织同步工具接口。

正常入口只包含可控、确定性的本地实验。故意保留的静态类型错误位于
static_errors.py；那个文件只交给 Pyright 分析，不作为程序执行。
"""

# 建议按文件从上往下读：先认识泛型值与搜索协议，再看工具注册表和 ABC，
# 最后由 main() 依次调用六组 demonstrate_* 函数，把前面“只定义、尚未执行”的代码跑一遍。
# 主数据流是：本地/fixture 数据 -> SearchBackend -> SearchTool -> Result[list[SearchHit]]。
# 类型注解负责把这条关系告诉 Pyright；真正运行时仍是普通 Python 对象、属性查找和函数调用。

# json 与 Path 共同读取本章固定的本地 fixture；ABC/abstractmethod 负责抽象基类约束；
# dataclass 生成数据类样板方法；collections.abc 中的类型描述调用、只读映射和只读序列能力；
# typing 中的 Generic/TypeVar 演示旧泛型语法，其余名称服务于协议、收窄和运行时浅检查。
import json
from abc import ABC, abstractmethod
from collections.abc import Callable, Mapping, Sequence
from dataclasses import dataclass
from pathlib import Path
from typing import Generic, Protocol, TypeVar, cast, runtime_checkable


def print_section(title: str) -> None:
    """让终端输出与 README 的实验顺序对应。"""
    # f-string 会先求值花括号里的 title，再拼出字符串；开头的 \n 先留一个空行分隔实验。
    # -> None 是静态返回提示，不会在运行时自动检查这个函数是否真的返回 None。
    print(f"\n=== {title} ===")


# Python 3.12+ 的类型参数语法把“这个 T 属于谁”直接写在函数名旁边。
# T 不是“任意值”的别名，而是一次调用中需要保持一致的类型变量：参数和返回值共享同一个 T。
def identity[T](value: T) -> T:
    """输入是什么具体类型，静态返回类型就保持为什么类型。"""
    # 这里直接返回传入对象本身，不复制、不转换，也不按注解做运行时校验。
    # 例如传入 int 时 Pyright 推断结果仍是 int；运行时只是让返回值引用同一个 int 对象。
    return value

```

```python
def first[T](values: Sequence[T]) -> T:
    """把容器元素类型与返回类型关联起来；空输入没有可返回的 T。"""
    # Sequence[T] 表示可按索引读取的一组 T；这里只读长度和首项，所以 list 与 tuple 都可传入。
    # 空 list/tuple 在布尔上下文中为 False。若为空，就没有真实元素可以兑现返回类型 T。
    if not values:
        raise ValueError("first() 不能接收空序列")
    # 经过上面的保护后取下标 0；返回类型仍与序列的元素类型绑定，而不是退化成 object/Any。
    return values[0]


# @dataclass 在类定义时加工类，为字段生成 __init__、__repr__ 和比较方法。
# frozen=True 禁止实例创建后重新给字段赋值，但不保证字段内部引用的可变对象也深度不可变。
@dataclass(frozen=True)
class Success[T]:
    """成功分支保存具体的 T，不把结果退化成 object 或 Any。"""

    # 构造 Success(15) 时 value 保存真实的 int；T 让静态检查器把该类型继续传给调用方。
    value: T


@dataclass(frozen=True)
class Failure:
    """失败分支保存原异常对象，调用方仍能看到异常类型和消息。"""

    # 保存异常对象而非仅保存字符串，后续仍可检查 ValueError/OSError 等具体类型。
    error: Exception

# type 声明的是带类型参数的类型别名：Result[int] 表示 Success[int] 或 Failure。
# 它不会额外包一层运行时对象；实际值仍是两个 dataclass 之一，可用 isinstance 分支判断。
type Result[T] = Success[T] | Failure


# 旧源码会先在模块级创建 TypeVar，再让类显式继承 Generic[T]。
# TypeVar 创建供类型检查器关联多个位置的类型变量；它不会限制随后赋给 _value 的运行时对象。
LegacyT = TypeVar("LegacyT")
```

```python
class LegacyBox(Generic[LegacyT]):  # noqa: UP046 - 本章需要展示旧框架源码写法
    """Python 3.11 及更早版本也能解析的传统泛型写法。"""

    def __init__(self, value: LegacyT) -> None:
        # self 指当前实例；单下划线表达“类内部使用”的约定，不是 Java private 的强制访问控制。
        self._value: LegacyT = value

    def get(self) -> LegacyT:
        # 构造参数与返回注解复用 LegacyT，因此 LegacyBox[str].get() 静态上仍返回 str。
        return self._value


# 现代语法把 T 的作用域收在 Box 类内，并隐式把该类声明为泛型类，不再继承 Generic[T]。
class Box[T]:
    """同一个 Box 关系的 Python 3.12+ 写法；本项目以它为主。"""

    def __init__(self, value: T) -> None:
        self._value: T = value

    def get(self) -> T:
        # 与旧写法一样，类型关系只服务于静态分析；运行时返回的就是先前保存的对象引用。
        return self._value


# SearchHit 是后端统一产出的数据记录，四个字段都由调用者按属性读取。
# dataclass 不会根据 str 注解做运行时校验；fixture 路径会在构造它之前显式验证外部数据。
@dataclass(frozen=True)
class SearchHit:
    document_id: str
    title: str
    snippet: str
    source: str


# SearchRequest 把工具输入收拢成一个对象；省略 limit 时，生成的 __init__ 会使用不可变整数 3。
@dataclass(frozen=True)
```

```
.10
111  # SearchRequest 把工具输入收拢成一个对象；省略 limit 时，生成的 __init__ 会使用不可变整数 3。
112  @dataclass(frozen=True)
113  class SearchRequest:
114      query: str
115      limit: int = 3
116
117
118  # 普通 Protocol 主要供静态检查。装饰器在类定义时把它标为可放进 isinstance() 的运行时协议；
119  # 这种运行时检查仍只看所需成员是否存在，不核对注解、完整参数签名或业务结果。
120  @runtime_checkable
121  class SearchBackend(Protocol):
122      """调用方只要求同步 search 能力，不要求实现类继承本协议。"""
123
124      # `*` 后的 limit 只能按名字传入，如 search("Protocol", limit=2)，不能作为第二个位置参数。
125      # `...` 是协议里的占位方法体：这里声明调用形状，不提供给隐式实现类复用的代码。
126      # 返回 Sequence 表示调用方只依赖读取能力；实现可返回能力更具体的 list[SearchHit]。
127      def search(self, query: str, *, limit: int) -> Sequence[SearchHit]: ...
128
129
130  class MemorySearchBackend:
131      """普通类没有显式继承 SearchBackend，但结构满足协议。"""
132
133      def __init__(self, documents: Sequence[SearchHit]) -> None:
134          # 构造器只读取输入，因此接收 list/tuple；tuple(...) 建立浅层快照，
135          # 避免调用方随后增删原序列的元素。元素对象本身不会被深拷贝。
136          self._documents: tuple[SearchHit, ...] = tuple(documents)
137                                                  I
138      def search(self, query: str, *, limit: int) -> list[SearchHit]:
139          # casefold() 做适合无大小写匹配的 Unicode 归一，split() 再按空白拆成查询词列表。
140          # 例如 "Protocol ABC" 会得到 ["protocol", "abc"]。
141          terms: list[str] = query.casefold().split()
142          # 注解说明这个可变列表只收集 SearchHit；Python 运行时不会凭注解阻止放入其他对象。
143          matches: list[SearchHit] = []
144          # 每轮 document 都是构造器快照中的一个 SearchHit。
145          for document in self._documents:
146              # f-string 把标题和摘要拼成一个待搜索字符串，再统一大小写以便比较。
```

```
130  class MemorySearchBackend:
138      def search(self, query: str, *, limit: int) -> list[SearchHit]:
145          for document in self._documents:
146              # f-string 把标题和摘要拼成一个待搜索字符串，再统一大小写以便比较。
147              searchable: str = f"{document.title} {document.snippet}".casefold()
148              # 生成器逐个产生“查询词是否出现”的布尔值；all() 遇到首个 False 就停止。
149              # 只有所有查询词都存在才收录。若查询词为空，all(空序列) 按定义为 True。
150              if all(term in searchable for term in terms):
151                  # append 原地修改 matches，加入的是现有 SearchHit 引用，不会复制该对象。
152                  matches.append(document)
153          # 切片返回新 list，原始 matches 与内部文档快照不变。经 SearchTool 调用时 limit 已为正数，
154          # 所以这里最多取前 limit 项；若绕过工具直接传负数，Python 会按负下标切片规则解释它。
155          return matches[:limit]
156
157
158  def require_text(mapping: Mapping[str, object], key: str, *, row: int) -> str:
159      """对本地 fixture 做最小运行时校验，避免用 cast 冒充已验证数据。"""
160      # Mapping 只承诺按键读取；get() 在缺键时返回 None，因此 value 先按宽泛 object 处理。
161      value: object | None = mapping.get(key)
162      # or 会短路：不是 str 时不会继续调用 value.strip()；是 str 时再拒绝仅含空白的内容。
163      # isinstance 分支也让静态检查器在成功路径上把 value 收窄为 str。
164      if not isinstance(value, str) or not value.strip():
165          # f-string 中 !r 使用 repr 形式展示 key，例如 'title'，便于定位坏字段。
166          raise ValueError(f"fixture 第 {row} 行的 {key!r} 必须是非空字符串")
167      # strip() 返回去掉两端空白的新字符串，不会原地修改 JSON 解码得到的值。
168      return value.strip()
169
170
171  def load_fixture(path: Path) -> tuple[SearchHit, ...]:
172      """读取并校验本章自带 JSON；这里只复刻本章实际需要的四个字段。"""
173      # with 类似 Java try-with-resources：离开缩进块时关闭文件，即使 json.load() 抛异常也一样。
174      # encoding 明确按 UTF-8 解码本章包含中文的 fixture。
175      with path.open(encoding="utf-8") as file:
176          # json.load() 把文本解析成 Python 的 list/dict/str 等对象。先标成 object，
177          # 表示外部数据尚未验证，不能提前假定它就是 list[SearchHit]。
178          decoded: object = json.load(file)
179
```

```python
def load_fixture(path: Path) -> tuple[SearchHit, ...]:
    # 顶层必须是 JSON 数组对应的 list；失败就停止，避免在错误形状上继续遍历。
    if not isinstance(decoded, list):
        raise TypeError("fixture 顶层必须是数组")

    # 逐行验证成功后才构造领域对象，最后再转成不可增删的 tuple 返回。
    documents: list[SearchHit] = []
    # enumerate(..., start=1) 同时给出从 1 开始的展示行号和当前原始元素，并解包到两个变量。
    for row, raw_item in enumerate(decoded, start=1):
        # JSON object 应解码为 dict；all(...) 再逐个确认键都是 str。
        # or 短路保证 raw_item 不是 dict 时，右侧不会错误地把它当字典迭代。
        if not isinstance(raw_item, dict) or not all(isinstance(key, str) for key in raw_item):
            raise ValueError(f"fixture 第 {row} 行必须是字符串键对象")

        # 上面的分支已经逐个证明键是 str；cast 只把已验证事实告知检查器。
        # cast 不转换、不复制，也不会在运行时再次检查 raw_item。
        item: Mapping[str, object] = cast(Mapping[str, object], raw_item)
        # 四次 require_text 分别取值并校验。只有全部成功才创建 SearchHit 并追加到 documents；
        # 任一字段失败会直接抛异常，不会留下一个字段不完整的 SearchHit。
        documents.append(
            SearchHit(
                document_id=require_text(item, "document_id", row=row),
                title=require_text(item, "title", row=row),
                snippet=require_text(item, "snippet", row=row),
                source=require_text(item, "source", row=row),
            )
        )
    # tuple(...) 建立返回容器；调用方能顺序读取命中，但不能对这个容器 append/remove。
    return tuple(documents)


class FixtureSearchBackend:
    """从本地 fixture 建立后端；对外仍只有同一个 search 协议。"""

    def __init__(self, path: Path) -> None:
        # 组合而非继承：先加载并验证 fixture，再把实际匹配工作委托给已实现的内存后端。
```

```python
209
210 class FixtureSearchBackend:
211     """从本地 fixture 建立后端；对外仍只有同一个 search 协议。"""
212
213     def __init__(self, path: Path) -> None:
214         # 组合而非继承：先加载并验证 fixture，再把实际匹配工作委托给已实现的内存后端。
215         self._delegate = MemorySearchBackend(load_fixture(path))
216
217     def search(self, query: str, *, limit: int) -> list[SearchHit]:
218         # 对外签名与 SearchBackend 一致；关键字参数显式转交给 delegate。
219         return self._delegate.search(query, limit=limit)
220
221
222 class SignatureMismatchBackend:
223     """教学反例：成员名存在，但注解签名与 SearchBackend 不一致。"""
224
225     # 协议要求 query: str、limit: int，这里故意反写。Pyright 会比较并拒绝这种结构兼容性。
226     def search(self, query: int, *, limit: str) -> list[SearchHit]:
227         # 函数体只把参数插入字符串，所以动态调用传入 str/int 恰好也能走通。
228         # 这不改变其声明的参数类型与协议冲突的事实。
229         return [
230             SearchHit(
231                 document_id="misleading-1",
232                 title=f"运行时检查只看成员：{query}",
233                 snippet=f"limit 的运行时值是 {limit!r}",
234                 source="signature-mismatch-demo",
235             )
236         ]
237
238
239 def load_runtime_candidate() -> object:
240     """模拟动态加载：调用方在运行前只知道候选对象是 object。"""
241     # 返回注解故意隐藏具体类，模拟插件/配置加载边界，让后面的 isinstance 分支真正承担收窄。
242     return SignatureMismatchBackend()
243
244
245 class OfflineSearchBackend:
```

```python
class OfflineSearchBackend:
    """可控失败替身：不联网，只同步抛出预期的 OSError。"""

    def search(self, query: str, *, limit: int) -> Sequence[SearchHit]:
        # raise 会中断当前调用，把异常交给外层 SearchTool 的 except；!r 让查询词带引号显示。
        raise OSError(f"本地后端暂不可用: query={query!r}, limit={limit}")


# 两个类型参数分别贯穿输入位置和成功输出位置：它们可以不同，但各自出现处必须保持关系。
class Tool[InputT, OutputT](Protocol):
    """输入类型、调用参数与 Result 中的成功类型形成一条关系。"""

    # 定义 __call__ 后，实例或普通函数只要签名兼容，就可当 Tool 使用。
    # `/` 之前的 tool_input 只能按位置传入；Result[OutputT] 把成功值类型继续传到调用端。
    def __call__(self, tool_input: InputT, /) -> Result[OutputT]: ...


class SearchTool:
    """把任意合规 SearchBackend 适配为 Tool[SearchRequest, list[SearchHit]]。"""

    def __init__(self, backend: SearchBackend) -> None:
        # 字段只依赖协议，不依赖 Memory/Fixture/Offline 的具体类，便于替换后端。
        self._backend: SearchBackend = backend

    def __call__(self, request: SearchRequest, /) -> Result[list[SearchHit]]:
        # 业务规则需要显式运行时判断；SearchRequest.limit 的 int 注解本身不会完成校验。
        # 非正数直接变成 Failure 数据，不调用后端，也不在这里抛给调用方。
        if request.limit < 1:
            return Failure(ValueError("limit 必须大于 0"))

        try:
            # 协议只承诺 Sequence；list(...) 将任意合规返回序列物化为工具拥有的具体列表。
            hits: list[SearchHit] = list(self._backend.search(request.query, limit=request.limit))
        except (OSError, ValueError) as error:
            # 圆括号列出两类预期边界异常，as error 把当前原始异常对象绑定到局部变量。
            # 只把已预期的边界失败转成 Result；原异常对象没有被替换或吞掉。
            return Failure(error)
```

```
262 class SearchTool:
269     def __call__(self, request: SearchRequest, /) -> Result[list[SearchHit]]:
275         try:
276             # 协议只承诺 Sequence；list(...) 将任意合规返回序列物化为工具拥有的具体列表。
277             hits: list[SearchHit] = list(self._backend.search(request.query, limit=request.limit))
278         except (OSError, ValueError) as error:
279             # 圆括号列出两类预期边界异常，as error 把当前原始异常对象绑定到局部变量。
280             # 只把已预期的边界失败转成 Result；原异常对象没有被替换或吞掉。
281             return Failure(error)
282         # 只有验证和后端调用都成功才到这里；类型检查器推断 Success 的 T 是 list[SearchHit]。
283         return Success(hits)
284
285
286 def empty_search_tool(request: SearchRequest, /) -> Result[list[SearchHit]]:
287     """普通函数也能满足带 __call__ 的 Tool 协议。"""
288     # 与 SearchTool 保持相同输入规则和 Result 形状，证明满足协议不要求包装成某个类。
289     if request.limit < 1:
290         return Failure(ValueError("limit 必须大于 0"))
291     # [] 是本次调用新建的空 list；Success 的目标类型由函数返回注解保持为 list[SearchHit]。
292     return Success([])
293
294
295 class ToolRegistry[InputT, OutputT]:
296     """小型同构注册表：同一实例中的工具共享一组输入/输出类型。"""
297
298     def __init__(self, tools: Mapping[str, Tool[InputT, OutputT]]) -> None:
299         # 公共参数只需读取，所以接收 Mapping；调用者可传 dict 或其他只读映射实现。
300         # dict(tools) 做浅复制，让注册表拥有可增删键的容器；工具对象本身仍与输入映射共享。
301         self._tools: dict[str, Tool[InputT, OutputT]] = dict(tools)
302
303     def register(self, name: str, tool: Tool[InputT, OutputT]) -> None:
304         # 下标赋值原地更新内部 dict；同名键会被新工具覆盖。
305         # 当前实例的 InputT/OutputT 同时约束初始工具、后续注册工具和 invoke() 结果。
306         self._tools[name] = tool
307
308     def invoke(self, name: str, tool_input: InputT) -> Result[OutputT]:
309         # get() 在名字不存在时返回 None，注册表值类型不包含 None，所以可据此判断缺失
```

```
295  class ToolRegistry[InputT, OutputT]:
308      def invoke(self, name: str, tool_input: InputT) -> Result[OutputT]:
309          # get() 在名字不存在时返回 None。注册表值类型不包含 None，所以可据此判断缺失。
310          tool: Tool[InputT, OutputT] | None = self._tools.get(name)
311          if tool is None:
312              # 未知名字也是预期调用结果，用 Failure 保存 LookupError，而非让 KeyError 直接冒出。
313              return Failure(LookupError(f"未注册工具：{name}"))
314          # 分支通过后 tool 被收窄为 Tool[InputT, OutputT]；像函数一样调用其 __call__。
315          return tool(tool_input)
316
317
318  # 类型别名把“一个位置参数、返回 str”的固定回调形状起成业务名称。
319  type HitFormatter = Callable[[Sequence[SearchHit]], str]
320
321
322  def format_titles(hits: Sequence[SearchHit]) -> str:
323      """简单位置参数回调可用 Callable 精确描述。"""
324      # 生成器逐项惰性产生 title，join()、再用顿号拼成一个 str，不创建中间标题列表。
325      # 空序列会拼出空字符串；空字符串为假，因此 or 返回右侧的展示占位文本。
326      return "、".join(hit.title for hit in hits) or "（空结果）"
327
328
329  def apply_formatter(hits: Sequence[SearchHit], formatter: HitFormatter) -> str:
330      # formatter 可以是函数、带 __call__ 的实例或其他可调用对象；这里按 Callable 约定调用。
331      return formatter(hits)
332
333
334  class PrefixedFormatter(Protocol):
335      """__call__ 协议能写出关键字参数名和调用规则。"""
336
337      # 基础 Callable 适合位置参数形状；协议还能明确 prefix 必须作为同名关键字传入。
338      # 实现无需继承本类，但 Pyright 会比较参数名称、位置规则和返回类型。
339      def __call__(self, hits: Sequence[SearchHit], *, prefix: str) -> str: ...
340
341
342  def format_with_prefix(hits: Sequence[SearchHit], *, prefix: str) -> str:
343      # 这个普通函数的签名恰好满足 PrefixedFormatter：f-string 将前缀与格式化结果连接。
```

```python
def format_with_prefix(hits: Sequence[SearchHit], *, prefix: str) -> str:
    # 这个普通函数的签名恰好满足 PrefixedFormatter；f-string 将前缀与格式化结果连接。
    return f"{prefix}{format_titles(hits)}"


# ABC 走名义继承：子类必须明确继承 ManagedTool，才能复用状态和 run() 模板流程。
class ManagedTool(ABC):
    """ABC 用继承提供共享流程，并阻止缺少实现的子类实例化。"""

    def __init__(self) -> None:
        # 每次构造实例都会得到独立的整数计数；_ 前缀仍只是内部使用约定。
        self._call_count = 0

    @property
    def call_count(self) -> int:
        # @property 让调用方写 managed.call_count 读取值，而不是 managed.call_count()。
        # 没有定义 setter，所以公共接口只读；内部仍可给 _call_count 重新赋值。
        return self._call_count

    def run(self, payload: Mapping[str, object]) -> str:
        # 这是所有子类共享的模板流程：先累计尝试次数，再分派到子类 execute()。
        # 计数发生在调用前，所以 execute() 即使抛异常，这次尝试也已经被计入。
        self._call_count += 1
        return self.execute(payload)

    # 装饰器在类创建时登记抽象成员。它不像普通类型注解那样仅供静态工具使用：
    # 仍未实现 execute() 的子类会在“实例化”时被 ABCMeta 以 TypeError 拒绝。
    @abstractmethod
    def execute(self, payload: Mapping[str, object]) -> str:
        """子类必须实现核心动作；共享的 run() 仍由基类提供。"""


class UppercaseManagedTool(ManagedTool):
    # 方法名和签名覆盖抽象成员，因此这个具体子类可以实例化。
    def execute(self, payload: Mapping[str, object]) -> str:
        # Mapping[str, object] 只保证键为 str、值是某种对象；get 缺键时也会得到 None。
```

```python
class UppercaseManagedTool(ManagedTool):
    # 方法名和签名覆盖抽象成员，因此这个具体子类可以实例化。
    def execute(self, payload: Mapping[str, object]) -> str:
        # Mapping[str, object] 只保证键为 str、值是某种对象；get 缺键时也会得到 None。
        text: object | None = payload.get("text")
        # ABC 只约束类结构，不验证 payload 数据，因此边界值仍需在运行时收窄。
        if not isinstance(text, str):
            raise TypeError("payload.text 必须是字符串")
        # str 不可变；upper() 返回新字符串，不会修改传入映射中的原值。
        return text.upper()


class IncompleteManagedTool(ManagedTool):
    """故意没有实现 execute，因此仍是抽象类。"""


# 这个最小协议只描述“可读取、结果为 str 的 title 属性”，不要求对象属于某个具体模型类。
class HasTitle(Protocol):
    # 用 property 描述属性式读取；调用方写 selected.title，而不是 selected.title()。
    @property
    def title(self) -> str: ...


# `T: HasTitle` 是上界：T 可以是任意满足 HasTitle 的更具体类型，且返回时保留该具体类型。
def first_titled[T: HasTitle](values: Sequence[T]) -> T:
    """bound 允许使用 title 能力，同时保留调用者的具体元素类型。"""
    # first() 把元素类型 T 带到 selected；上界又保证函数体能安全读取 selected.title。
    selected: T = first(values)
    # 空字符串在布尔上下文中为 False；这里只校验标题非空，不改变 selected。
    if not selected.title:
        raise ValueError("title 不能为空")
    return selected


# `A: (str, bytes)` 是有限约束（constraint）：每次调用从候选中求出一个共同的 A。
# 两个参数不能一个选 str、另一个选 bytes；传入子类时，静态结果会归一到匹配的候选家族。
```

```python
# `A: (str, bytes)` 是有限约束 (constraint)：每次调用从候选中求出一个共同的 A。
# 两个参数不能一个选 str、另一个选 bytes；传入子类时，静态结果会归一到匹配的候选家族。
def combine_text[A: (str, bytes)](left: A, right: A) -> A:
    """约束要求两个位置选择同一个 str/bytes 家族，并保留该家族结果。"""
    # 相同类型关系保证 + 对本次家族有定义：str 做文本拼接，bytes 做字节拼接。
    return left + right


def passthrough_union(value: str | bytes) -> str | bytes:
    """普通 union 只说单个位置可为两者之一，不建立输入与输出的同型关系。"""
    # 运行时仍原样返回对象；差异在静态信息：返回注解只承诺 str | bytes 这个可能集合。
    return value


def append_debug_marker(values: list[object]) -> None:
    """能接收 list[object] 的函数有权插入任意 object。"""
    # list 可变，append() 会原地污染调用者持有的同一个列表。这正是 list 元素类型不变的原因：
    # 若把 list[SearchHit] 放宽成 list[object]，这里合法追加的 str 会破坏原列表的元素保证。
    values.append("debug-marker")


def demonstrate_generics() -> None:
    # 本组先用普通值观察 T 怎样连接参数与返回值，再比较 Result 的两个分支和新旧泛型类语法。
    print_section("1. 泛型表达多个位置之间的关系")

    # 两次调用运行时都只是普通函数调用；静态分析分别把 T 求解为 int 和 str。
    # 元组的类型可读作“任意长度的 str 序列”，first() 再把元素类型带到返回值。
    number: int = identity(15)
    title: Literal['Protocol'] | Literal['ABC'] | Li = first(("Protocol", "ABC", "Generic"))
    # type(value).__name__ 读取实际运行时类名，用来对照类型检查器推断和真实对象是否一致。
    print("identity(15) 保留值/类型:", number, type(number).__name__)
    print("first(tuple[str, ...]) 返回 str:", title, type(title).__name__)

    # 空 tuple 写作 ()。first() 无法产生 T，于是抛 ValueError；这里只捕获预期异常，
    # 让实验继续执行，同时打印失败原因，而不是把所有 Exception 都吞掉。
    try:
```

```
429 def demonstrate_generics() -> None:
441     # 空 tuple 写作 ()。first() 无法产生 T，于是抛 ValueError；这里只捕获预期异常，
442     # 让实验继续执行，同时打印失败原因，而不是把所有 Exception 都吞掉。
443     try:
444         first(())
445     except ValueError as error:
446         print("空序列没有可返回的 T:", error)
447
448     # 两个变量都声明为 Result[int]，但运行时实际分别装着 Success[int] 和 Failure。
449     # Failure 不需要 int 值：它保存的是这次失败的原异常对象。
450     success: Result[int] = Success(number)
451     failure: Result[int] = Failure(ValueError("保留的原始失败"))
452     # isinstance 根据实际 dataclass 分支收窄联合；Success 分支中 value 因而仍按 int 使用。
453     if isinstance(success, Success):
454         print("Result[int] 成功值仍可做整数计算:", success.value + 1)
455     # Failure 分支可以同时观察异常类名和异常消息，证明没有只留下一个错误字符串。
456     if isinstance(failure, Failure):
457         print("失败分支保留异常:", type(failure.error).__name__, failure.error)
458
459     # 两个 Box 都由实参推断元素类型为 str；get() 把各自保存的值原样返回。
460     # 输出相同能力是为了说明这里只更换了泛型声明语法，没有更换运行时容器行为。
461     legacy: LegacyBox[str] = LegacyBox("旧语法")
462     modern: Box[str] = Box("3.12+ 主写法")
463     print("传统 TypeVar/Generic 与新语法:", legacy.get(), "|", modern.get())
464
465
466 def demonstrate_protocol_backends() -> tuple[SearchBackend, SearchBackend]:
467     # 本组构造两个没有继承 SearchBackend 的类，再让同一段调用代码通过协议类型使用它们。
468     print_section("2. duck typing 与可替换的 SearchBackend")
469
470     # 外层 tuple 是传给内存后端的初始只读序列；其中每项都是 frozen dataclass SearchHit。
471     # 这两条数据故意分别覆盖 Protocol 和 ABC 查询，方便观察匹配结果。
472     memory_backend = MemorySearchBackend(
473         (
474             SearchHit(
475                 document_id="memory-1",
476                 title="Protocol 入门"
```

```python
def demonstrate_protocol_backends() -> tuple[SearchBackend, SearchBackend]:
    memory_backend = MemorySearchBackend(
        (
            SearchHit(
                document_id="memory-1",
                title="Protocol 入门",
                snippet="结构满足即可替换实现。",
                source="memory",
            ),
            SearchHit(
                document_id="memory-2",
                title="ABC 生命周期",
                snippet="继承基类获得共享流程。",
                source="memory",
            ),
        )
    )
    # __file__ 是当前源码路径；with_name() 只替换文件名，因此不依赖启动程序时的工作目录。
    # FixtureSearchBackend 会立即读取、校验 JSON，并在内部建立另一个 MemorySearchBackend。
    fixture_path: Path = Path(__file__).with_name("search_fixture.json")
    fixture_backend = FixtureSearchBackend(fixture_path)

    # 两个类都没写“继承 SearchBackend”，但方法结构兼容，仍可放进协议类型的同一个 tuple。
    # 这份注解是给 Pyright 的静态契约；运行时 tuple 只保存两个普通实例引用。
    backends: tuple[SearchBackend, SearchBackend] = (memory_backend, fixture_backend)
    # 每轮 backend 指向其中一个实现；调用者不检查类名，也不知道数据来自内存还是 JSON。
    for backend in backends:
        hits: list[SearchHit] = backend.search("Protocol", limit=2)
        # 列表推导式从每个 SearchHit 读取 document_id，生成只用于展示的新 str 列表。
        print(type(backend).__name__, "->", [hit.document_id for hit in hits])

    # runtime_checkable 让 isinstance 可运行；列表推导式分别得到两个布尔值。
    # True 只证明 search 成员存在，不证明完整签名与行为，下一组实验专门展示这个边界。
    print("两种实现都通过运行时成员检查:", [isinstance(item, SearchBackend) for item in backends])
    # 把两个后端交给后续注册表实验继续复用，具体类型在返回注解中抽象为协议。
    return backends
```

```python
def demonstrate_runtime_protocol_limit() -> None:
    # 本组把“运行时能看到成员”与“静态签名真正兼容”故意拆开，避免把浅检查当类型验证。
    print_section("3. runtime_checkable 不检查完整签名或业务行为")

    # 候选值的静态类型先视为 object，isinstance 分支才会按 runtime-checkable 协议做类型收窄。
    candidate: object = load_runtime_candidate()
    # 实际对象确实有名为 search 的成员，所以尽管参数注解完全相反，这里仍得到 True。
    passed: bool = isinstance(candidate, SearchBackend)
    print("错误注解签名仍通过 isinstance:", passed)
    if passed:
        # True 分支内，类型检查器把 candidate 收窄成 SearchBackend，允许按协议写法调用。
        # 运行时只做普通参数绑定，不执行注解校验；反例函数体又只格式化参数，所以碰巧能执行。
        hits: Sequence[SearchHit] = candidate.search("Protocol", limit=1)
        # 这里有一项固定反例结果，因此安全取下标 0；它只能证明当前假实现走通这一条路径。
        print("动态调用碰巧走通:", hits[0].title)
    # 同一个对象赋给 SearchBackend 的静态反例放在不可执行文件中，避免正常演示因故意错误失败。
    print("静态赋值为何失败请运行 Pyright 检查 static_errors.py")


def print_tool_result(label: str, result: Result[list[SearchHit]]) -> None:
    # Success/Failure 是可区分的运行时类；先判断成功分支，也同步帮助 Pyright 收窄联合。
    if isinstance(result, Success):
        # value 的静态类型是 list[SearchHit]，运行时再展示其真实类名和元素数量。
        print(f"{label}: 成功，具体值类型={type(result.value).__name__}，数量={len(result.value)}")
    else:
        # Result 只有两个成员，排除 Success 后此处就是 Failure，可以直接访问原异常 error。
        print(f"{label}: 失败，{type(result.error).__name__}: {result.error}")


def demonstrate_registry(backends: tuple[SearchBackend, SearchBackend]) -> list[SearchHit]:
    # 本组把协议后端包装成统一 Tool，并让同构注册表调度五条可观察的成功/失败路径。
    print_section("4. Tool[Input, Output]、注册表与 Result[Output]")

    # 映射中所有值共享 Tool[SearchRequest, list[SearchHit]] 这一组类型参数：
    # 前两个是可调用类实例，第三个是普通函数，结构兼容使它们可放进同一注册表。
    tools: Mapping[str, Tool[SearchRequest, list[SearchHit]]] = {
```

```python
def demonstrate_registry(backends: tuple[SearchBackend, SearchBackend]) -> list[SearchHit]:
    # 映射中所有值共享 Tool[SearchRequest, list[SearchHit]] 这一组类型参数：
    # 前两个是可调用类实例，第三个是普通函数，结构兼容使它们可放进同一注册表。
    tools: Mapping[str, Tool[SearchRequest, list[SearchHit]]] = {
        "memory_search": SearchTool(backends[0]),
        "fixture_search": SearchTool(backends[1]),
        "empty_search": empty_search_tool,
    }
    # 构造器会复制映射；随后注册的离线工具只在 registry 内部 dict 中增加一个名字。
    registry: ToolRegistry[SearchRequest, list[Searc... = ToolRegistry(tools)
    registry.register("offline_search", SearchTool(OfflineSearchBackend()))

    # 同一个不可变请求对象送给每个工具，便于只比较实现路径差异。
    request = SearchRequest(query="Protocol", limit=2)
    # 五次 invoke 都返回相同静态类型 Result[list[SearchHit]]；运行时分支分别是：
    # 两个真实命中、一个空成功、一个保存 OSError 的失败、一个保存 LookupError 的失败。
    memory_result: Success[list[SearchHit]] | Failure = registry.invoke("memory_search", request)
    fixture_result: Success[list[SearchHit]] | Failure = registry.invoke("fixture_search", request)
    empty_result: Success[list[SearchHit]] | Failure = registry.invoke("empty_search", request)
    offline_result: Success[list[SearchHit]] | Failure = registry.invoke("offline_search", request)
    missing_result: Success[list[SearchHit]] | Failure = registry.invoke("missing", request)

    # 每个二元组是（展示标签，Result）；for 在每轮按位置解包为 label 和 result。
    # 统一交给 print_tool_result 后，一个失败不会中止剩余场景。
    for label, result in (
        ("内存工具", memory_result),
        ("fixture 工具", fixture_result),
        ("普通函数工具", empty_result),
        ("受控后端失败", offline_result),
        ("未知工具", missing_result),
    ):
        print_tool_result(label, result)

    # 后续 Callable/ABC 实验需要实际命中，因此只从 fixture 的成功分支取值。
    if isinstance(fixture_result, Success):
        # Result 的 T 是 list[SearchHit]，分支收窄后无需 Any/cast 就能取 SearchHit。
        selected: SearchHit = first(fixture_result.value)
```

```python
def demonstrate_registry(backends: tuple[SearchBackend, SearchBackend]) -> list[SearchHit]:

    # 后续 Callable/ABC 实验需要实际命中，因此只从 fixture 的成功分支取值。
    if isinstance(fixture_result, Success):
        # Result 的 T 是 list[SearchHit]，分支收窄后无需 Any/cast 就能取 SearchHit。
        selected: SearchHit = first(fixture_result.value)
        print("成功分支首项:", selected.document_id, selected.title)
        # 返回的是 Success 内保存的同一个 list 引用，没有在此复制。
        return fixture_result.value
    # fixture 是仓库内固定合法数据；走到这里说明示例自身假设被破坏，应作为程序错误停止。
    raise AssertionError("内置 fixture 应产生成功结果")


def demonstrate_callable_and_abc(hits: Sequence[SearchHit]) -> None:
    # 本组先比较两种"可调用对象"签名，再观察 ABC 如何复用状态并拒绝不完整子类实例化。
    print_section("5. Callable、__call__ Protocol 与 ABC")

    # format_titles 是普通函数，签名满足 HitFormatter，apply_formatter 通过参数间接调用它。
    print("Callable[[Sequence[SearchHit]], str]:", apply_formatter(hits, format_titles))
    # 普通函数也能结构化满足 __call__ 协议；变量注解要求它保留 prefix 这个关键字名。
    formatter: PrefixedFormatter = format_with_prefix
    print("__call__ 协议保留关键字规则:", formatter(hits, prefix="命中: "))

    # 具体 ABC 子类继承 __init__、run 和 property；run 内再动态分派到覆盖后的 execute。
    managed = UppercaseManagedTool()
    print("ABC 共享 run 流程:", managed.run({"text": "agent tool"}))
    # 上一次 run 已在调用 execute 前把实例计数从 0 增至 1。
    print("ABC 共享调用次数:", managed.call_count)

    # 不完整子类的"类定义"本身允许存在；真正调用类对象尝试创建实例时才抛 TypeError。
    try:
        # 此处故意触发运行时规则；行尾指令只压制 Pyright 对已知抽象类用法的静态诊断。
        IncompleteManagedTool()  # pyright: ignore[reportAbstractUsage]
    except TypeError as error:
        # 只捕获预期的实例化错误，并展示类型；对象没有创建成功，任何方法体也没有运行。
        print("未实现 abstractmethod 不能实例化:", type(error).__name__)

```

```python

def demonstrate_bounds_constraints_and_variance(hits: list[SearchHit]) -> None:
    # 本组用同一批 SearchHit 对照 bound、constraint、union，再故意演示可变列表的类型污染。
    print_section("6. bound、约束、union 与可变容器")

    # hits 的元素满足 HasTitle；bound 允许读取 title，同时把具体 SearchHit 类型带回 selected。
    selected: SearchHit = first_titled(hits)
    print("bound 保留 SearchHit:", selected.document_id, selected.title)
    # 两次合法调用分别让受约束的 A 统一选择 str 家族和 bytes 家族。
    print("约束让两个位置同为 str:", combine_text("tool-", "result"))
    print("约束让两个位置同为 bytes:", combine_text(b"tool-", b"result"))
    # 普通 union 的实际值仍是传入的 str，但签名只表达返回值属于两种可能之一。
    print("union 只声明可能集合:", passthrough_union("query"))

    # [:1] 浅复制一个只含首项的新 list：容器与 hits 分离，内部 SearchHit 引用仍共享。
    contaminated: list[SearchHit] = hits[:1]
    # 这行故意越过 list 的静态不变性；局部副本让反例不会污染原 hits。
    # 行尾指令仅关闭这一行的 Pyright 参数诊断，不会改变运行时调用。
    append_debug_marker(contaminated)  # pyright: ignore[reportArgumentType]
    # 推导式显示污染后两个元素的实际类名，稳定结果是 SearchHit 与 str。
    print("若放行 list[SearchHit] -> list[object]:", [type(item).__name__ for item in contaminated])
    # Sequence 不提供 append，适合只读消费者；要修改时才应请求准确元素类型的 list。
    print("因此只读公开参数优先 Sequence；需要追加时才索取 list")


def main() -> None:
    """按从最小泛型到完整工具接口的顺序运行所有同步实验。"""
    # 前三组定义/验证基础关系；第二组返回的两个协议后端会继续流入注册表。
    demonstrate_generics()
    backends: tuple[SearchBackend, SearchBackend] = demonstrate_protocol_backends()
    demonstrate_runtime_protocol_limit()
    # 注册表返回 fixture 的命中列表，最后两组把它作为回调输入和泛型/方差样本继续使用。
    hits: list[SearchHit] = demonstrate_registry(backends)
    demonstrate_callable_and_abc(hits)
    demonstrate_bounds_constraints_and_variance(hits)


```

```
610  def demonstrate_bounds_constraints_and_variance(hits: list[SearchHit]) -> None:
623      # [:1] 浅复制一个只含首项的新 list：容器与 hits 分离，内部 SearchHit 引用仍共享。
624      contaminated: list[SearchHit] = hits[:1]
625      # 这行故意越过 list 的静态不变性；局部副本让反例不会污染原 hits。
626      # 行尾指令仅关闭这一行的 Pyright 参数诊断，不会改变运行时调用。
627      append_debug_marker(contaminated)  # pyright: ignore[reportArgumentType]
628      # 推导式显示污染后两个元素的实际类名，稳定结果是 SearchHit 与 str。
629      print("若放行 list[SearchHit] -> list[object]:", [type(item).__name__ for item in contaminated])
630      # Sequence 不提供 append，适合只读消费者；要修改时才应请求准确元素类型的 list。
631      print("因此只读公开参数优先 Sequence；需要追加时才索取 list")
632
633
634  def main() -> None:
635      """按从最小泛型到完整工具接口的顺序运行所有同步实验。"""
636      # 前三组定义/验证基础关系；第二组返回的两个协议后端会继续流入注册表。
637      demonstrate_generics()
638      backends: tuple[SearchBackend, SearchBackend] = demonstrate_protocol_backends()
639      demonstrate_runtime_protocol_limit()
640      # 注册表返回 fixture 的命中列表，最后两组把它作为回调输入和泛型/方差样本继续使用。
641      hits: list[SearchHit] = demonstrate_registry(backends)
642      demonstrate_callable_and_abc(hits)
643      demonstrate_bounds_constraints_and_variance(hits)
644
645
646  # 导入本模块时，Python 会创建上面的函数与类，但 __name__ 是模块名，因此不会自动跑实验。
647  # 直接执行 example.py 时 __name__ 才等于 "__main__"，于是从 main() 开始按顺序产生六组输出。
648  if __name__ == "__main__":
649      main()
650
```

```
[
  {
    "document_id": "fixture-1",
    "title": "Protocol 让工具后端可替换",
    "snippet": "调用方依赖 search 能力，不依赖具体存储方式。",
    "source": "local-fixture"
  },
  {
    "document_id": "fixture-2",
    "title": "实用泛型保留输入输出关系",
    "snippet": "Result[T] 让成功数据保持具体类型。",
    "source": "local-fixture"
  },
  {
    "document_id": "fixture-3",
    "title": "ABC 管理固定生命周期",
    "snippet": "抽象方法阻止不完整子类实例化。",
    "source": "local-fixture"
  }
]

```

```python
"""只供 Pyright 分析的第 15 章反例；不要执行本文件。

这里故意保留类型错误和 reveal_type()，用来对照运行时成员检查与静态签名检查。
"""

from collections.abc import Sequence
from typing import reveal_type

from example import (
    Box,
    PrefixedFormatter,
    Result,
    SearchBackend,
    SearchHit,
    SearchRequest,
    SignatureMismatchBackend,
    Success,
    Tool,
    append_debug_marker,
    combine_text,
    first,
    first_titled,
    identity,
    passthrough_union,
)

# 诊断 1：runtime_checkable 只看 search 成员是否存在；静态协议会比较完整签名。
bad_backend: SearchBackend = SignatureMismatchBackend()


# 诊断 2：identity 把参数类型和返回类型绑定为同一个 T，不会“允许任意后再变成 str”。
wrong_identity_result: str = identity(15)


# 诊断 3：first 的返回类型来自 Sequence 的元素类型。
numbers: tuple[int, ...] = (1, 2, 3)
wrong_first_result: str = first(numbers)
```

```python

def wrong_output_tool(request: SearchRequest, /) -> Result[str]:
    return Success(request.query)


# 诊断 4：Tool 的 Output 同时约束 __call__ 返回的 Result[Output]。
search_tool: Tool[SearchRequest, list[SearchHit]] = wrong_output_tool


def wrong_keyword_name(hits: Sequence[SearchHit], label: str) -> str:
    return label + str(len(hits))


# 诊断 5：协议调用会传 prefix=，但实现只声明了另一个关键字名 label。
bad_formatter: PrefixedFormatter = wrong_keyword_name


# 诊断 6：list 可变；若允许这次赋值，函数就能把 str 插入 list[SearchHit]。
hit_list: list[SearchHit] = []
append_debug_marker(hit_list)


# 诊断 7：约束 TypeVar 要求多个位置选择同一个允许类型，不能混用 str 与 bytes。
mixed_text: str = combine_text("tool-", b"result")


class SpecialQuery(str):
    pass


special_query = SpecialQuery("Protocol")
reveal_type(combine_text(special_query, special_query))  # Pyright: str
reveal_type(passthrough_union(special_query))  # Pyright: str | bytes


special_hit = SearchHit("special", "特殊命中", "bound 示例", "local")
reveal_type(first_titled([special_hit]))  # Pyright: SearchHit
```

```
54
55
56  # 诊断 6：list 可变；若允许这次赋值，函数就能把 str 插入 list[SearchHit]。
57  hit_list: list[SearchHit] = []
58  append_debug_marker(hit_list)
59
60
61  # 诊断 7：约束 TypeVar 要求多个位置选择同一个允许类型，不能混用 str 与 bytes。
62  mixed_text: str = combine_text("tool-", b"result")
63
64
65  class SpecialQuery(str):
66      pass
67
68
69  special_query = SpecialQuery("Protocol")
70  reveal_type(combine_text(special_query, special_query))  # Pyright: str
71  reveal_type(passthrough_union(special_query))  # Pyright: str | bytes
72
73
74  special_hit = SearchHit("special", "特殊命中", "bound 示例", "local")
75  reveal_type(first_titled([special_hit]))  # Pyright: SearchHit
76  reveal_type(Box(special_hit).get())  # Pyright: SearchHit
77
```